

07. 검증 · DFT · MBIST 완전 이해 — "설계가 맞다는 걸 어떻게 증명하나"

정독용 심층 해설서. 외우는 카드가 아니라 왜 이런 검증 체계가 생겼는가를 1st-principles로 끝까지 따라가는 문서다. 대상: 삼성 S.LSI에서 RTL Sign-off 방법론·검증 자동화·정합성(Formality EQ)·AI 검증을 4년차로 한 뒤 SK하이닉스 메모리 설계(설계검증 / HBM Digital Design 검증 / NAND Logic) 신입 지원자. 이 주제는 지원자의 **핵심 강점 영역**이다. 그래서 다른 개념서보다 더 깊고 서사적으로 간다. 연계: 기초는 `daily_study/H_verif.md · G_digital.md`, 경력 방어는 `04_resume_selfmastery.md`, 질문은 `03_interview_job_qbank.md` 의 C4~C8.

0. 이 문서의 지도 — 검증이라는 거대한 영역을 한 장으로

"설계를 했다"는 것과 "설계가 맞다"는 것은 완전히 다른 문장이다. RTL을 짜는 데는 한 달이 걸려도, 그 RTL이 모든 상황에서 사양대로 동작한다는 것을 증명하는 데는 그 두세 배가 걸린다. 현대 SoC·메모리 개발에서 **검증 (verification)**이 전체 설계 공수의 절반 이상을 차지한다는 말은 과장이 아니라 산업 통계다. 그 이유를 이 문서 전체가 설명한다.

먼저 큰 지도를 그리자. "설계가 맞다"를 증명하는 도구는 크게 세 갈래다. 이 셋의 역할 분담을 처음부터 명확히 머리에 박아두면, 뒤의 모든 세부가 제자리를 찾는다.

"설계가 맞다"를 증명하는 세 축			
기능 검증 (Functional Verif.)	정적 타이밍 검증 (STA)	정합성/형식 검증 (Formal / LEC)	
질문: "기능이 맞나?"	"이 주파수에 맞나?"	"두 설계가 같나?" / "이 속성이 항상 참?"	
방법: 시뮬레이션 (자극을 넣고 관찰)	그래프 기반 정적 분석 (벡터 없음, 전수 경로)	수학적 증명 (입력 전수, 벡터 없음)	
대상: 동작 · 시나리오	setup/hold · slack	RTL↔netlist, protocol	
한계: 본 것만 안다	기능은 안 봄	큰 데이터패스엔 폭발	
도구: SV/UVM	PrimeTime 등	Formality/JasperGold	
↑ 동적(자극 필요)	↑ 정적	↑ 정적(exhaustive)	

핵심 통찰 하나: **시뮬레이션은 "본 곳만" 보장하고, formal/STA는 "전수"를 본다.** 시뮬레이션에 입력 벡터를 1억 개 넣어도 그것은 우주의 한 점이다. 반면 STA는 모든 타이밍 경로를 그래프로 전수 분석하고, formal은 모든 입력 조합을 수학적으로 증명한다. 그래서 좋은 검증 엔지니어는 "이 문제는 시뮬레이션이 맞나, formal이 맞나, STA가 맞나"를 먼저 가르다. 지원자가 삼성에서 한 일 — Lint/CDC/RDC sign-off, Formality EQ — 은 정확히 이 **정적·형식 검증 축**의 한복판이었다. 메모리 설계검증으로 옮겨도 이 지도는 그대로다.

1. 왜 검증이 설계비용의 절반 이상인가 — 1st-principles

이것부터 "왜?"로 풀어야 한다. 면접에서 "검증이 왜 그렇게 중요합니까"를 물으면, 답은 **상태공간 폭발**과 **silicon respin**의 **비대칭 비용** 두 가지다.

(a) **상태공간은 지수적으로 폭발한다.** n 개의 플립플롭이 있는 설계는 이론상 2^n 개의 상태를 가진다. FF가 1,000개만 되어도 2^{1000} — 우주의 원자 수보다 많다. 시뮬레이션으로 이 공간을 "다 돌아본다"는 것은 물리적으로 불가능하다. 그래서 검증은 "다 본다"를 포기하고 **"중요한 곳을, 체계적으로, 측정 가능하게 본다"**로 전략을 바꿨다. coverage라는 개념, constrained-random이라는 기법, formal이라는 우회로가 전부 이 폭발에 대한 대응이다.

(b) **버그를 늦게 잡을수록 비용이 지수적으로 커진다.** RTL 단계에서 버그를 잡으면 코드 한 줄 고치면 된다. 합성 후에 잡으면 다시 합성·STA를 돌려야 한다. silicon이 나온 뒤에 잡으면 — 마스크 한 세트 다시 만들고(수억~수십억 원), 몇 달의 일정이 날아가고(respin), 이미 출하된 칩이면 리콜이다. 메모리처럼 **양산 물량이 막대한 제품**은 이 비대칭이 더 극단적이다. 그래서 "왼쪽으로 이동(shift-left)" — 가능한 한 앞 단계에서, 가능한 한 형식적으로 잡는 것 — 이 모든 검증 방법론의 철학이다.

지원자의 경험으로 이 철학을 증명할 수 있다. **RTL Sign-off Agent로 false violation 20%↓**, **RDC false violation 47,159→5,795건(87.7%↓)**, **SFR-to-HW CDC noise 95% 자동분류** — 이 숫자들은 전부 "설계자가 silicon 전에, 더 빠르고 정확하게 버그/위험을 숙아내게 만든" 일이다. 검증의 본질이 "shift-left + 비용 절감"임을 몸으로 한 사람이라는 뜻이다.

2. RTL → 게이트, 흐름의 전 구간 — 각 단계에서 무엇이 바뀌고 무엇을 검증하나

검증을 이해하려면 **무엇을 검증하는지**, 즉 설계물이 어떤 변환을 거치는지를 알아야 한다. RTL 한 줄이 silicon이 되기까지의 여정과, 각 변환 직후에 끼어드는 검증 관문을 그려보자.

```
RTL (사람이 짠 동작 기술)
|
| ← Lint(코딩 규칙) · CDC/RDC(도메인 경계) · 기능 시물(UVM) · SVA/Formal
▼
[ 논리 합성 Synthesis ]   RTL → gate-level netlist (표준셀로 매핑)
|
| ← LEC(RTL ≡ netlist 정합성) · 합성 후 STA(타이밍 1차)
▼
[ DFT 삽입 ]   scan chain · MBIST · compression 회로를 netlist에 넣음
|
| ← LEC(DFT 삽입 후에도 기능 보존?) · ATPG(패턴 생성 · fault coverage)
▼
[ P&R Place & Route ]   배치 · 배선, clock tree(CTS), 실제 지연 결정
|
| ← Sign-off STA(실 지연 · OCV) · LEC(ECO 후 정합성) · 물리검증(DRC/LVS)
▼
[ Tape-out → 제조 → 웨이퍼 ]
|
| ← EDS/wafer test: scan/ATPG 패턴 인가, MBIST 실행, repair(BISR)
▼
[ 패키지 → 양산 ]
```

각 화살표가 "무엇이 바뀌었으니 무엇을 다시 검증해야 하나"를 말한다. 여기서 결정적인 통찰 두 개:

첫째, 합성은 "기능이 같은, 다른 구조"로 바꾼다. RTL의 $a + b$ 는 합성기가 ripple-carry로도, carry-lookahead로도 매핑할 수 있다. 모양은 완전히 달라지지만 기능은 같아야 한다. *같아야 한다*를 보장하는 것이 바로 **LEC(정합성 검증)** — 6장에서 깊게 다룬다. `G_digital.md` 에서 정리한 "동작(시물) RTL ≠ 합성 후 동일 보장 아님" (incomplete sensitivity list, 의도치 않은 latch 추론, blocking/non-blocking 오용)이 정확히 이 지점의 위험이고, lint + LEC가 그 방패다.

둘째, DFT 삽입은 "기능과 무관한 회로를 일부러 끼워 넣는" 단계다. scan chain은 평소 기능 동작에는 관여하지 않지만 제조 후 테스트를 위해 존재한다. 끼워 넣었으니 "끼워 넣어도 원래 기능이 망가지지 않았나"를 다시 LEC로 본다. DFT가 검증 흐름의 정식 일부인 이유가 여기 있다.

이 흐름 전체가 **sign-off** — "이 단계를 통과했으니 다음으로 넘긴다"는 책임 있는 서명들의 연쇄 — 다. 지원자가 "RTL Sign-off 방법론"을 했다는 것은 이 관문들을 *설계자가 신뢰하고 통과할 수 있게 만드는 체계*를 만들었다는 뜻이다.

3. 기능 검증의 패러다임 전환 — directed → constrained-random → coverage-driven

이제 세 축 중 가장 큰 덩어리, 기능 검증으로 들어간다. 여기엔 **세 번의 패러다임 전환**이 있었고, 그 역사를 이해하면 UVM이 왜 그렇게 생겼는지가 저절로 보인다.

3.1 1세대: directed test와 그 벽

초기 검증은 사람이 시나리오를 하나씩 손으로 짰다. "리셋 후 명령 A를 보내면 응답이 와야 한다" — 이런 테스트를 directed test라 한다. 직관적이고, 특정 버그를 콕 집어 재현하기 좋다. 그러나 벽이 있다. **사람이 생각한 경우만 본다**. 1장의 상태공간 폭발 앞에서, 사람의 상상력으로 corner case를 다 나열하는 것은 불가능하다. 두 비동기 이벤트가 *정확히 같은 사이클에* 겹치는 희귀한 충돌 같은 것을 사람이 미리 다 적을 수는 없다.

3.2 2세대: Constrained-Random Verification(CRV)

발상의 전환은 이것이었다. "사람이 케이스를 나열하지 말고, **유효한 제약 안에서 자극을 무작위로 생성**하게 하자." 제약(constraint)이란 "주소는 0~1023 범위", "프로토콜상 명령은 ACT 다음에만 RD" 같은 **유효성 규칙**이다. 그 안에서 난수로 자극을 쏟아 부으면, 사람이 미처 생각 못 한 조합에 자동으로 도달한다. SystemVerilog는 `rand / constraint` 로 이걸 언어 차원에서 지원한다.

하지만 즉시 새 문제가 생긴다. 자극을 무작위로 던졌으니, **무엇이 옳은 출력인지를 사람이 미리 알 수 없다**. directed test는 "이 입력엔 이 출력"이 정해져 있었지만, random은 그렇지 않다. 그래서 두 가지가 반드시 따라온다: (1) 무엇이 옳은지를 독립적으로 판정하는 **reference model + scoreboard**, (2) 옳은지 그른지를 시간 축에서 감시하는 **assertion**. 이 둘이 없는 CRV는 "자극만 많고 판정은 없는" 무의미한 시뮬레이션이다.

`H_verif.md` 의 핵심 문장 — "constrained random=제약 내 랜덤으로 corner 자동 도달, but 무엇이 옳은지는 scoreboard/reference가 판단" — 이 정확히 이 구조다.

3.3 3세대: Coverage-Driven Verification — "얼마나 봤는지"를 측정하다

random을 돌리면 새 질문이 생긴다. "**충분히 돌렸나?** 언제 멈추나?" 무작정 오래 돌린다고 끝이 아니다. 같은 곳만 반복해서 때리고 있을 수도 있다. 그래서 **coverage** — "지금까지 무엇을, 얼마나 봤는가"를 측정하는 계량기 — 가

도입됐다. 그리고 그 계량기를 피드백 삼아 "아직 안 본 곳"을 향해 자극을 조정하는 것이 coverage-driven verification이다.

여기서 두 종류의 coverage를 반드시 구분해야 한다. 이건 면접 단골이다.

	Code Coverage (구조적)	Functional Coverage (의도적)
질문	"코드를 실행은 했나?"	"의도한 시나리오가 일어났나?"
추출	도구가 RTL에서 자동	설계자가 covergroup 으로 수동 정의
항목	line / branch / condition / toggle / FSM state-transition	coverpoint(특정 값) / cross(조합)
한계	"실행만" 했지 의미는 모름	정의 안 한 것은 측정도 안 됨

핵심 명제: **code coverage 100% ≠ 검증 완료**. 왜? code coverage는 "모든 줄을 한 번씩 밟았다"만 말한다. 하지만 "캐시가 가득 찬 동시/에 인터럽트가 들어오고 동시/에 refresh가 겹치는" 특정 조합을 한 번도 안 만들었다면, 그 줄들은 각각은 실행됐어도 그 교차 상황은 본 적이 없다. 그 교차를 명시적으로 정의하는 것이 functional coverage의 cross다. 그래서 둘 다 필요하다 — code는 "코드를 다 건드렸나", functional은 "의도한 상황을 다 봤나".

또 하나 절대 헛갈리면 안 되는 것: coverage는 "안 본 곳"을 알려줄 뿐, "본 곳이 맞는지"는 보장하지 않는다. coverage 100%여도 scoreboard-assertion이 없으면 버그를 못 잡는다. coverage는 완전성(completeness)의 척도이고, checker는 정확성(correctness)의 척도다. 둘은 다른 축이다.

3.4 Coverage Closure — sign-off의 실제 의미

그래서 기능 검증의 종료(sign-off)는 단순히 "테스트가 통과했다"가 아니라 **coverage closure**다. 구체적으로:

1. **code + functional coverage가 목표치 도달** (보통 100%, 혹은 합의된 수준).
2. **모든 assertion pass** — 시간 축의 모든 속성 위반 0.
3. **regression이 안정적으로 green** — 수백·수천 개 테스트를 매일 자동으로 돌려(야간 regression) 회귀(이전에 고친 버그의 재발)가 없음을 지속 확인.
4. **남은 coverage hole이 전부 설명됨** — 못 채운 칸은 (a) directed test를 추가로 짜서 채우거나, (b) **도달 불가(unreachable)임을 formal/분석으로 입증**해 waiver(예외 처리)한다. "그냥 안 채워진 채로 넘김"은 없다.

이 "hole을 채우거나, 도달 불가를 입증해 waiver한다"는 사고는 지원자의 CDC/RDC waiver 관리 철학과 **완전히 같은 구조**다. RDC에서 "확실히 안전한 것만 깎고 애매하면 남긴다"는 원칙, CDC에서 "unreachable를 입증해 exclusion"하는 것 — coverage hole waiver와 동일한 보수적 sign-off 철학이다. 메모리 검증에서 "이 corner는 왜 안 채웠나"를 물으면, 지원자는 이 waiver 관리 경험으로 정확히 답할 수 있다.

 **메모리 브릿지:** 메모리 컨트롤러·HBM Base Die의 프로토콜 검증은 명령 조합(ACT/RD/WR/PRE)·timing corner(tRCD, tRP...)·bank 경합·refresh 충돌이 거대한 cross coverage 공간을 만든다. refresh가 두 bank에서 동시/에 트리거되는 회귀 충돌 같은 것은 directed로는 못 만들고 constrained-random으로 자동 발굴해야 한다. 그리고 그 거대한 coverage 공간을 자동으로 분석하고 hole을 추적하는 것이 바로 지원자가 한 검증 자동화의 가치다.

4. UVM 완전 이해 — 왜 "표준"이고, 무엇으로 이루어졌나

CRV + coverage-driven을 체계적이고 재사용 가능하게 구현하려면 골격이 필요하다. 그 골격의 산업 표준이 **UVM(Universal Verification Methodology)** 이다. SystemVerilog의 OOP(class·상속·다형성) 위에 세워진 검증 라이브러리·방법론이다.

4.1 왜 표준이 필요했나 — 재사용의 경제학

UVM 이전에는 회사·팀마다 검증 환경을 제각기 짰다. 문제는 **재사용 불가**였다. A팀이 만든 testbench를 B팀이 못 가져다 썼고, 같은 IP를 다음 프로젝트에서 또 처음부터 짰다. 검증이 비싸지는 핵심 원인이 "매번 새로 짜기"였다. UVM의 존재 이유는 한 단어, **재사용성**이다. 표준화된 구조·인터페이스·생성 메커니즘(factory)을 두면, 한 번 검증한 IP의 환경을 *VIP(Verification IP)* 형태로 묶어 다음 프로젝트·다른 팀에 그대로 넘길 수 있다. 검증 자산이 부채가 아니라 재고가 된다.

4.2 계층 구조 — layered testbench

UVM 환경은 책임을 계층으로 나눈다. 위에서 아래로:

```
test          ← 어떤 시나리오를 돌릴지 선택 (최상위)
├─ env        ← 검증 환경 전체를 담는 컨테이너
│   ├─ agent  ← 하나의 인터페이스를 담당하는 단위 (재사용의 핵심 블록)
│   │   ├─ sequencer  sequence를 arbitration해 driver에 전달
│   │   ├─ driver     transaction을 DUT 핀으로 구동 (interface 통해)
│   │   └─ monitor    DUT 핀을 수동 관찰해 transaction 복원
│   └─ scoreboard  monitor가 복원한 값 vs reference model 기대값 비교 → pass/fail
└─ coverage    monitor 출력으로 functional coverage 수집
```

각 부품의 역할을 왜 그렇게 나눴는지와 함께 보자:

- **sequence / sequence_item**: 자극의 "내용"을 데이터로 표현한다. sequence_item은 한 트랜잭션(예: "주소 0x40에 0xDEAD를 write"), sequence는 그런 item들의 시나리오(흐름)다. **자극을 코드가 아니라 데이터로** 만들었기 때문에, 같은 환경에 sequence만 같아 끼워 무한히 다른 테스트를 만들 수 있다. directed든 random이든 sequence 레벨에서 결정된다.
- **sequencer**: 여러 sequence가 동시에 자극을 보내려 할 때 누구를 먼저 보낼지 arbitration(중재)하고, 정리된 item을 driver에 넘긴다.
- **driver**: 받은 추상 트랜잭션을 실제 *핀 wiggling*으로 번역한다. "write item" → 핀에 주소를 싣고, write enable을 올리고, 클럭 맞춰 데이터를 내보내는 프로토콜. driver는 **프로토콜을 아는 유일한 곳**이라, 프로토콜이 바뀌면 여기만 고친다.
- **monitor**: driver의 반대 방향. 핀을 수동으로 관찰해서, 거기서 일어난 일을 다시 추상 트랜잭션으로 복원한다. 절대 구동하지 않고 보기만 한다(그래서 active/passive 무관하게 항상 존재). 이 복원된 트랜잭션이 scoreboard와 coverage로 흘러간다.
- **scoreboard**: 정확성의 심판. monitor가 본 DUT의 실제 출력과, 독립적으로 계산한 *reference model의 기대값*을 비교한다. 둘이 다르면 버그. 3.2절에서 말한 "random은 정답을 미리 모른다" 문제를 이 reference model이 푼다.
- **agent**: sequencer+driver+monitor를 묶은 **재사용 단위**. *active agent*는 셋 다 갖고 구동까지 하고, *passive agent*는 monitor만 뒤서 관찰·coverage 전용이다(다른 곳이 이미 구동 중일 때). 인터페이스 하나당 agent

하나가 원칙이라, 새 인터페이스가 생기면 agent를 하나 더 붙인다.

4.3 재사용을 떠받치는 두 메커니즘 — factory와 config_db

UVM이 단순 라이브러리가 아니라 *방법론*인 이유는 **factory**와 **config_db** 때문이다.

- **factory**: 객체를 `new` 로 직접 만들지 않고 factory를 통해 만든다. 그러면 환경 코드를 한 줄도 안 고치고 "이 driver를 저 자식 driver로 바꿔치기(override)"가 가능하다. error injection 버전 driver로 교체, 특정 테스트만 다른 monitor 사용 — 전부 factory override로 한다.
- **config_db**: 설정을 계층 위에서 아래로 주입한다. 환경의 동작 모드, virtual interface 핸들 등을 상위에서 내려보내고 하위가 받는다. 부품이 자기 설정을 하드코딩하지 않으니, 같은 부품을 다른 설정으로 재사용할 수 있다.

이 둘 덕분에 *환경의 뼈대는 그대로 두고, 행동만 바꾸는* 것이 가능하다. 이것이 "재사용 가능한 검증 환경"의 실체다.

4.4 "Spec에서 Custom UVM Agent를 A-to-Z로 만든다"는 게 무슨 뜻인가

SK하이닉스 설계 JD에 반복 등장하는 문구가 "**Spec 기반 Custom UVM Agent**" 다. 이게 무슨 작업인지 구체적으로 풀어보자. 표준 프로토콜(AXI, APB 등)은 벤더 VIP를 사다 쓰면 된다. 그런데 메모리는 **사내 고유의 인터페이스·프로토콜**이 많다 — 특정 메모리 명령 시퀀스, 커스텀 테스트 모드 인터페이스, HBM Base Die 내부 버스 같은 것. 이런 건 시중에 VIP가 없다. 그래서 그 인터페이스의 *spec(타이밍 다이어그램·핸드셰이크 규칙·신호 정의)*을 읽고, 그것을 구동·관찰·검사하는 UVM agent를 **처음부터 직접 설계**해야 한다:

1. **spec 정독** → 인터페이스의 transaction 단위(무엇이 한 거래인가)와 프로토콜(어떤 순서·타이밍 규칙)을 추출.
2. **sequence_item 정의** — transaction을 클래스 필드로 모델링(rand 필드 + constraint로 유효 범위).
3. **driver 작성** — item을 spec대로 핀 프로토콜로 번역.
4. **monitor 작성** — 핀에서 transaction 복원 + 프로토콜 위반 체크(SVA 결합).
5. **scoreboard·coverage 연결** — 기대값 모델, coverpoint/cross 정의.
6. **agent로 패키징** → env에 꽂아 재사용 가능한 VIP로.

지원자에게 이걸 **새로 배울 영역이 아니라, 이미 한 일의 다른 표현**이다. SFR 자동생성에서 IP-XACT spec으로부터 검증환경(register access test·기본값/속성 체크 같은 UVM류 환경 골격)을 자동생성한 경험(04 문서 11번)이 바로 "spec → 검증환경 구축"의 정수다. spec을 단일 소스로 삼아 검증 인프라를 생성하는 사고를 이미 했고, DVCon에서 발표까지 한 사람이다. JD의 "Spec 기반 Custom UVM Agent"는 지원자가 이미 자동화까지 밀어붙였던 작업의 수작업 버전인 셈이다.

 **메모리 브릿지**: 메모리 컨트롤러 검증에서 host측은 트랜잭션을 구동하는 active agent로, DRAM/PHY측은 프로토콜을 관찰하는 passive agent로 둔다. JEDEC 프로토콜(DDR/HBM 명령·타이밍 규칙) 준수를 monitor 안의 SVA로 상시 감시한다. UVM의 layered 구조가 "복잡한 메모리 프로토콜을 부품 단위로 나눠 검증"하는 데 정확히 맞는 이유다.

5. SVA와 Formal — 의도를 선언하고, 전수로 증명하다

5.1 SVA: 버그를 "발생 지점에서, 시간 축으로" 잡는다

시뮬레이션의 고질병 하나: 버그가 내부에서 발생해도, 그게 출력 핀까지 전파되어 scoreboard에 잡힐 때까지 모른다. 그 사이 수백 사이클이 흐르면, 디버그할 때 "출력이 틀렸다"에서 "내부 어디가 원인이나"를 거슬러 올라가야 한다. 시간 낭비다.

SVA(SystemVerilog Assertion) 는 이걸 뒤집는다. "이 신호는 항상 이래야 한다", "req가 뜨면 2 사이클 안에 ack가 와야 한다" 같은 설계 의도를 선언적으로 박아두면, 위반이 발생하는 그 순간 그 자리에서 시뮬레이터가 잡아준다. 버그를 출력까지 추적할 필요가 없다. 디버그 시간이 극적으로 준다.

두 종류를 구분하자:

- **Immediate assertion**: 절차 코드(`always`) 안에서 그 순간의 조건을 즉시 체크. 조합적·1점 검사. (예: 함수 진입 시 인자 유효성)
- **Concurrent assertion**: 클럭에 동기화되어 여러 사이클에 걸친 시간적 속성을 검사. property/sequence로 기술한다. 이게 SVA의 진짜 힘이다.
- `a |-> b` : a가 참이면 같은 사이클에 b도 참 (overlapping implication).
- `a |=> b` : a가 참이면 다음 사이클에 b 참 (non-overlapping).
- 예: `req |-> ##[1:3] ack` — req 후 1~3 사이클 안에 ack. 메모리 명령 타이밍 규칙이 전형적으로 이렇게 표현된다.

또 SVA는 `cover property` 로 **coverage**에도 쓴다. "이 시퀀스(예: refresh 도중 read 충돌)가 한 번이라도 일어났나"를 functional coverage 항목으로 만든다. 즉 SVA는 checker이자 coverage 수집기다.

5.2 Formal: 시뮬레이션이 못 가는 곳을 수학으로 증명

SVA를 박아도, 시뮬레이션은 여전히 "내가 넣은 자극 하에서만" 그 assertion이 안 깨졌다고 말한다. 어떤 희귀한 입력 시퀀스가 그 assertion을 깰 수 있는데 내가 그 시퀀스를 안 넣었다면? 못 잡는다.

Formal property verification은 이 한계를 넘는다. 자극을 넣지 않고, 모든 가능한 입력 시퀀스에 대해 그 property가 참인지를 수학적으로 증명한다. 결과는 둘 중 하나: (1) **proven** — 어떤 입력으로도 절대 안 깨진다 (전수 증명), 또는 (2) **반례(counterexample)** — 그것을 깨는 구체적 입력 파형을 도구가 찾아준다. 시뮬레이션은 "내가 만든 자극에선 안 깨졌다"인데, formal은 "세상의 어떤 자극으로도 안 깨진다"다. 차원이 다른 보증이다.

bounded vs unbounded 구분도 중요하다: - **Bounded proof**: 리셋에서 N 사이클 안에서는 안 깨진다(깊이 제한). 빠르지만 "N 너머"는 보장 못 함. - **Unbounded(full) proof**: 무한 시간에 대해 증명. 가장 강하지만 상태공간이 크면 도구가 수렴 못 하고 폭발한다.

그래서 formal이 잘 맞는 문제 유형이 따로 있다 — 면접 단골이다. 상태공간이 작고 제어 중심인 블록: arbiter, FSM, 핸드셰이크 프로토콜, 인터럽트 컨트롤러, CDC 제어 로직 같은 것. 반대로 넓은 데이터패스(곱셈기, 큰 메모리 어레이)는 상태공간이 폭발해 formal이 어렵고, 거기선 시뮬레이션·LEC가 낫다. "어떤 문제에 formal을 쓰겠나"라는 질문엔 "작은 제어 블록·프로토콜에 unbounded, 데이터패스엔 시뮬/LEC" 가 정답이다.

🧠 **메모리 브릿지:** 지원자는 이미 formal을 했다 — CDC/RDC가 전형적인 formal-structural 검증 영역이고, SDC 검증에서 Xcelium TCV(formal 기반 타이밍 제약 검증)를 평가했다(04 문서 6번). 그리고 CSR spec에서 CDC constraint-assertion을 자동생성하는 PoC(04 문서 2번)는 "의도를 assertion으로 박는다"는 SVA 철학 그 자체다. 메모리에선 JEDEC 프로토콜 규칙, refresh/command 타이밍 제약, 작은 제어 FSM을 SVA+formal로 증명한다. "CDC 동기화 프로토콜-메모리 command 타이밍을 SVA로 명세-검증"이 지원자의 CDC 업무와 직결된다.

6. 정합성 검증(LEC) — "두 설계가 같다"를 수학으로 증명하다 (지원자 최강 영역)

이 장은 지원자의 Formality Quick/Full EQ 경험이 그대로 사는 곳이다. 깊게 간다.

6.1 왜 필요한가 — 변환의 연쇄가 만드는 위험

2장에서 봤듯, RTL은 silicon이 되기까지 합성·DFT 삽입·ECO·restructuring 등 여러 변환을 거친다. 각 변환은 "기능은 보존하면서 구조만 바꾼다"가 목표지만, 도구 버그·코딩 함정·휴먼 에러로 기능이 *미세하게* 달라질 수 있다. 이 차이는 시뮬레이션으로 잡기 어렵다 — 특정 입력에서만 드러나는데, 그 입력을 안 넣으면 모른다.

LEC(Logic Equivalence Check, 정합성/논리등가 검증)는 이걸 *형식적으로* 푼다. 두 설계(예: RTL vs 합성 netlist)가 기능적으로 완전히 동일함을 수학적으로 증명한다. 시뮬레이션이 아니다 — 자극을 안 넣는다. Synopsys Formality, Cadence Conformal이 대표 도구다.

6.2 어떻게 동작하나 — compare point와 logic cone

LEC의 작동 원리를 1st-principles로 보자. 순차 회로 전체를 한 번에 비교하면 상태공간이 폭발한다. 그래서 LEC는 회로를 **compare point**들로 쪼갬다. compare point란 비교의 기준점 — 레지스터(FF), primary output, black-box의 입력이다. 회로를 이 점들로 자르면, 점과 점 사이에는 *조합 논리만* 남는다. 한 compare point로 흘러드는 조합 논리 덩어리를 **logic cone**(논리 원뿔)이라 부른다.

그러면 비교는 이렇게 환원된다: "두 설계의 대응하는 compare point들을 매칭하고, 각 compare point로 모이는 logic cone이 모든 입력 조합에서 **같은 값을 내는지** 증명한다." 조합 논리의 등가는 SAT/BDD 같은 형식 기법으로 전수 증명할 수 있다. 순차 문제를 조합 문제들의 모음으로 분해한 것이 LEC의 핵심 아이디어다.

시뮬레이션 대비 결정적 우위: 시뮬레이션은 입력 벡터의 일부만 본다. LEC는 **모든 입력 조합을 형식적으로** 증명한다. 등가성을 보장해야 하는 문제에는 LEC가 정답이지 시뮬레이션이 아니다. (단, 6.5에서 보듯 큰 데이터패스는 형식 증명도 폭발할 수 있어 도구의 solver 전략이 중요하다.)

6.3 어디에 쓰나 — 변환마다 한 번씩

- **합성 후:** RTL $\equiv$ gate netlist. 합성기가 기능을 보존했는지.
- **DFT 삽입 후:** scan/MBIST를 끼워도 기능 동작이 보존됐는지(끼운 테스트 회로가 기능 모드에서 암전한지).
- **ECO 후:** 막판 수정(Engineering Change Order) netlist가 의도한 변경 외에는 그대로인지.
- **Restructuring 후:** RTL $\leftrightarrow$ RTL — hierarchy를 재구성한 결과가 원본과 같은지.

6.4 지원자의 실제 경험 서사 — ARM 코어 hardening의 Quick/Full EQ 이원화

여기가 지원자의 CPU IP Hardening(04 문서 4번) 경험이 빛나는 지점이다. 서사로 풀어보자.

ARM 최신 코어를 삼성 공정에 맞게 SoC compiler(Defacto)로 **재구조화(restructuring)** 하는 과제였다. restructuring을 하면 hierarchy가 자동으로 변형되는데, 이 과정에서 **parameter 전달 오류**, **\ define 소실**, **unrolling** 같은 이유로 변형된 RTL이 원본과 *미세하게* 달라질 위험이 있다. 시뮬레이션으로는 이런 미세 차이를 보장 못 한다. 그래서 Formality로 "**변형 RTL = 원본 RTL**"을 sign-off했다.

여기서 진짜 어려운 엔지니어링 디테일이 나온다. 재구조화로 인스턴스 경로(hierarchy path)가 바뀌므로, 원본의 어떤 레지스터가 변형본의 어떤 레지스터에 대응하는지 — 즉 **compare point 매칭** — 을 도구가 자동으로 못 한다. 수동으로 `set_compare_rule` 을 일일이 쓰면 휴먼 에러가 난다. 지원자는 SoC compiler의 mapping 파일을 파싱해 **compare rule**을 자동생성해서 휴먼 에러를 0으로 만들었다. 이건 "LEC를 돌렸다"가 아니라 "LEC가 돌아가게 만드는 인프라를 자동화했다"는, 한 단계 위의 작업이다.

그리고 Full/Quick EQ 이원화: - **Full EQ**: 전체 hierarchy·모든 compare point를 정밀 검증 → 최종 sign-off용. 느리지만 완전. - **Quick EQ**: 재구조화된 하위 계층을 **black-box**로 처리하고, 새 wrapper의 인터페이스 정합성 (port mismatch·방향 오류)만 빠르게 1차 확인 → **디버그 TAT 단축용**. 빠르지만 black-box 안은 안 본다.

이 이원화의 철학이 중요하다 — **속도와 정확도를 분리**한 것이다. Quick EQ로 연결 오류를 조기에 빠르게 숙고, 최종 무결성은 Full EQ로 보장한다. 이 사고는 4.x의 active/passive agent, coverage의 hole waiver와 같은 결의 "검증 자원을 현명하게 배분한다"는 sign-off 엔지니어의 성숙함이다. 이 환경으로 **약 1,760만 line 설계를, 인력이 5→2명으로 줄어든 상황에서도** 무결성을 검증했다.

발견한 실제 이슈도 서사로 가치가 있다: ① wrapper의 parameter type이 바뀌거나 fixed value로 변환되어 EQ가 fail → 벤더(Defacto) 개선요청 + workaround. ② 특정 SRAM instance에서 \ define 구문이 소실 → 재현해서 벤더 공식 bug fix를 유도. "EQ를 돌려 통과시켰다"가 아니라 "**도구의 버그까지 잡아내 벤더를 움직였다**" — formal 검증의 깊이를 보여주는 일화다.

6.5 면접 방어 포인트

- "Quick EQ에서 black-box하면 그 안은 검증 안 되는데?" → 맞다. Quick은 1차 연결 오류 조기검출용이고, 최종 sign-off는 Full EQ로 그 안까지 본다. 속도-정확도의 의도적 분리다.
- "non-equivalence가 나오면?" → failing compare point의 logic cone을 디버그한다. 원인이 도구 변형 버그면 벤더로, 의도된 차이면 compare rule/black-box 설정을 조정한다.
- "시뮬레이션과 뭐가 다른가?" → 시뮬은 입력 벡터 일부만, EQ는 모든 입력 조합을 형식적으로 증명. 등가성엔 EQ가 정답.

 **메모리 브릿지**: 메모리 Peri·HBM Base Die의 디지털 IP도 합성·ECO·재사용·DFT 삽입을 거친다. 그때마다 "기능이 보존됐나"를 LEC로 sign-off한다. 지원자의 Quick/Full EQ 이원화 + compare rule 자동생성 인프라는 메모리 Peri 정합성 검증에 그대로 이식된다. "1,760만 line을 인력 반 토막에도 검증한 자동화"는 양산 메모리의 방대한 IP를 적은 인력으로 sign-off해야 하는 SK하이닉스 현장에 직접 가치다.

7. DFT 완전 이해 — 제조 결함을 "테스트할 수 있게" 미리 설계하다

여기서 검증의 성격이 바뀐다. 1~6장은 전부 DV(Design Verification) — "설계가 사양대로 맞게 만들어졌나"를 silicon 전에 본다. 이제 DFT(Design for Test) — "제조된 chip이 물리적으로 제대로 만들어졌나"를 테스트하는 회로를 설계 단계에서 넣는 일 — 로 넘어간다. 같은 "검증"이라도 DV는 설계의 옳음, DFT는 제조의 옳음을 본다. (JD가 검증·DFT를 설계 직무의 핵심으로 두는 이유다.)

7.1 왜 테스트 용이성을 설계 때 넣어야 하나 — controllability와 observability

제조된 칩의 내부 깊은 곳 어떤 게이트가 stuck-at(0이나 1에 고착)됐다고 하자. 그걸 칩 바깥의 핀에서 테스트하려면 두 가지가 필요하다: (1) 그 게이트 입력을 원하는 값으로 만들 수 있어야 하고(controllability, 제어성), (2) 그 게이트 출력의 변화를 핀에서 볼 수 있어야 한다(observability, 관찰성). 그런데 일반 순차 회로에서 내부 FF는 수십 단계의 로직 뒤에 묻혀 있어, 핀에서 특정 내부 상태를 만들고 관찰하는 게 거의 불가능하다.

그래서 발상: 테스트하기 쉬운 구조를 설계 단계에서 미리 끼워 넣자. 제조 후에는 회로를 못 바꾸니, 설계 때 controllability/observability를 높여두는 것 — 이것이 DFT의 존재 이유다.

7.2 Scan chain — FF를 shift register로 변신시키다

핵심 기법이 scan이다. 일반 FF를 scan FF(앞에 MUX가 붙은 FF)로 바꾼다. 그리고 모든 scan FF를 한 줄로 shift register처럼 연결한다(scan chain). MUX의 선택 신호가 scan_enable이다.

두 가지 모드로 동작한다:

```
scan_enable = 1 (SHIFT 모드):
  scan_in → [FF]→[FF]→[FF]→ ... →[FF] → scan_out
  체인 전체가 거대한 shift register. 핀(scan_in)에서 원하는 비트열을
  클럭마다 밀어 넣어 모든 내부 FF를 임의 값으로 세팅(controllability 확보).
  동시에 이전에 캡처한 값을 scan_out으로 밀어내 핀에서 관찰(observability 확보).

scan_enable = 0 (CAPTURE 모드):
  FF들이 잠깐 정상 기능 모드로 동작.
  보통 1 클럭. 내부 조합 로직의 응답을 FF가 "캡처".
```

테스트 한 사이클의 흐름: shift-in(체인으로 테스트 입력 패턴을 밀어 넣음) → capture(scan_enable=0, 한두 클럭 기능 동작시켜 로직 응답을 FF에 캡처) → shift-out(다음 패턴을 shift-in하면서 동시에 캡처값을 shift-out해 핀에서 관찰). 이렇게 하면 핀 몇 개로 칩 내부 전체를 제어·관찰할 수 있다. 묻혀 있던 내부가 핀으로 끌려 나온 것이다.

7.3 ATPG와 fault model — 무엇을, 어떻게 테스트하나

scan으로 "내부를 제어·관찰"할 길은 뚫었다. 이제 무슨 패턴을 넣어야 결함을 잡나? 이것 자동으로 만드는 게 ATPG(Automatic Test Pattern Generation) 다(Tessent TestKompress, Synopsys TestMAX 등). ATPG는 fault model(결함 모델)을 가정하고, 각 가정된 결함을 드러내는 테스트 패턴을 생성한다.

대표 fault model: - **Stuck-at fault**: 어떤 노드가 영구히 0(stuck-at-0)이나 1(stuck-at-1)에 고착. 가장 기본. 잡으려면 그 노드를 반대값으로 만들고(controllability) 출력에서 관찰(observability)해 차이를 본다. -

Transition(delay) fault: 노드가 값을 *천이*하긴 하는데 *너무 느려서* 제때 못 바뀜. 이건 **at-speed test** — 실제 동작 주파수로 두 클럭을 빠르게 줘서(launch-capture) "제 시간에 천이했나"를 봐야 잡힌다. stuck-at은 느린 클럭으로도 잡지만, transition은 빠른 클럭이 필요하다. - (그 외 bridging, IDDQ 등.)

fault coverage: 가정한 전체 결함 중 ATPG 패턴이 실제로 *잡을 수 있는* 비율. 보통 99%+를 목표로 한다. 못 잡는 결함(undetectable)은 분석해서 설명한다 — 이 "fault coverage를 채우고 못 채운 건 설명한다"는 구조가 3.4의 coverage closure와 *완전히 같은 철학*임을 주목하라.

7.4 Scan compression — 테스트 시간·비용의 압박

문제: 칩이 커지면 FF가 수백만 개라 scan chain이 길어지고, 패턴이 많아져 **테스트 시간(=테스터 점유 시간=비용)**과 **테스트 데이터 용량**이 폭발한다. 메모리·로직 양산에서 테스트 시간은 곧 단가다. 그래서 **scan compression:** 적은 수의 외부 핀에서 들어온 압축된 자극을 칩 내부의 **decompressor**가 수많은 짧은 내부 체인으로 펼치고, 응답은 **compressor(MISR 등)**가 다시 압축해 적은 핀으로 내보낸다. 테스트 데이터·시간을 한 자릿수~두 자릿수 배 줄인다. (Tessent의 SSH/SSN — Streaming Scan Network — 같은 아키텍처가 이 압축·전송을 확장한 형태다.)

7.5 Scan과 클럭/CDC의 충돌 — 면접 단골

여기 *날카로운* 포인트가 있다. shift 모드에서는 **모든 scan FF가 하나의 scan 클럭으로 한 줄로 묶여** 직렬 이동한다. 그런데 원래 설계는 여러 클럭 도메인으로 나뉘어 있었다. 즉 **shift 동안에는 평소의 CDC 구조(2-FF synchronizer 등)가 무력화**되고, 서로 다른 도메인의 FF들이 한 체인에서 직렬로 움직인다. 그래서:

- shift 모드의 클럭킹은 *기능 모드의 CDC 가정을 깬다* → scan용 클럭 제어(여러 도메인을 안전하게 직렬 shift하도록)와, capture 시 도메인 간 경로 처리가 별도로 필요하다.
- 또 scan_enable·scan 클럭 같은 *테스트 제어 신호*는 high-fanout net이라 clock/reset 못지않게 신중히 다뤄야 한다(`G_digital.md`의 high-fanout 예시에 scan enable이 등장하는 이유).

이 "scan이 CDC와 충돌한다"를 이해하는 사람은 드물고, 지원자의 **CDC 전문성**과 DFT가 만나는 정확한 접점이다. 면접에서 "scan과 CDC의 충돌?"이 나오면, "shift 시 도메인 경계가 직렬 체인으로 묶여 기능 모드 동기화 구조가 무력화되므로, 별도 scan 클럭 제어와 capture 경로 처리가 필요하다 — 제 CDC sign-off 관점에서 이 경계 관리가 핵심"이라 답하면 강력하다.

 **메모리 브릿지:** 메모리는 array 결함률이 본질적으로 높아 **DFT 비중이 로직보다 크다**. Peri의 디지털 로직은 scan/ATPG로, array는 8장의 MBIST로 — 두 축으로 테스트한다. "DFT = 수율 직결, 메모리는 테스트가 곧 비용"이라는 명제가 이 장의 핵심이고, scan compression이 그 비용을 줄이는 무기다.

8. 가장 "메모리다운" 검증 — MBIST · Redundancy · Repair (C7의 정수)

7장의 scan/ATPG는 *로직*을 테스트한다. 그런데 메모리 array(수억~수십억 셀)는 핀에서 일일이 셀 하나하나를 못 본다. 또 셀 결함은 stuck-at 같은 단순 모델로 안 끝나고 *메모리 특유의 결함* 유형이 따로 있다. 그래서 **메모리는 메모리 전용 테스트 체계**를 가진다. 이것이 가장 "메모리스러운 검증"이고, 신입 지원자가 이걸 정확히 알면 *값을 메우는* 강점이 된다.

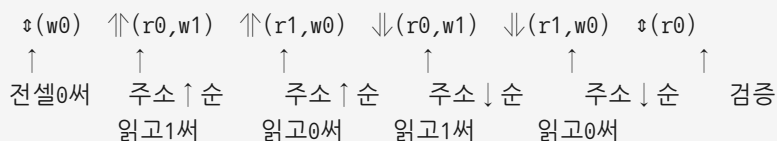
8.1 MBIST — 칩 스스로를 테스트하다

MBIST(Memory Built-In Self-Test) 는 칩 *내부에* 작은 테스트 엔진(BIST controller)을 넣어, 외부 테스터 없이 메모리 array를 스스로 진단하게 한다. 왜 self-test인가? (1) 외부 테스터로 고속 메모리를 풀 스피드로 때리는 건 비싸고 어렵다(테스터 대역폭 한계). (2) 적층된 HBM의 *내부 die*는 패키징 후 외부 핀에서 접근조차 어렵다. 칩이 스스로 테스트하면 이 둘을 푼다 — 외부 테스터 부담↓, at-speed 테스트 가능.

MBIST 엔진은 메모리에 정해진 패턴을 write/read하며 결함을 찾는데, 그 패턴 알고리즘의 대표가 **March 알고리즘**이다.

8.2 March 알고리즘 — 왜 그렇게 읽고 쓰나

March test는 메모리 전 주소를 *정해진 순서로 훑으며(march)* write/read를 반복하는 알고리즘 군이다. 예를 들어 March C-:



이 순서가 임의로 정해진 게 아니다. 각 단계가 특정 결함 유형을 *드러내도록* 설계됐다. March가 잡는 결함: - **Stuck-at fault(SAF)**: 셀이 0/1에 고착 → write 후 read에서 불일치. - **Transition fault(TF)**: 0→1이나 1→0 천이가 안 됨. - **Coupling fault(CF)**: 한 셀에 write가 *인접 셀*의 값을 바꿈(셀 간 간섭) → 주소를 오름/내림 양방향으로 훑어야 잡힘(그래서 ↑와 ↓ 둘 다 있다). - **Address decoder fault(AF)**: 주소 디코더가 엉뚱한 셀을 지목. - **Neighborhood pattern sensitive fault**: 주변 셀 패턴에 따라 특정 셀이 고장.

"March가 잡는 결함 유형은?"이라는 꼬리질문엔 **SAF/TF/CF/AF + 양방향 march로 coupling을 잡는다가** 정답이다. 셀 간 간섭(coupling)을 양방향 훑기로 잡는다는 디테일이 메모리스러움의 핵심이다.

8.3 Redundancy — 불량 셀을 여분으로 갈아끼워 수율을 산다

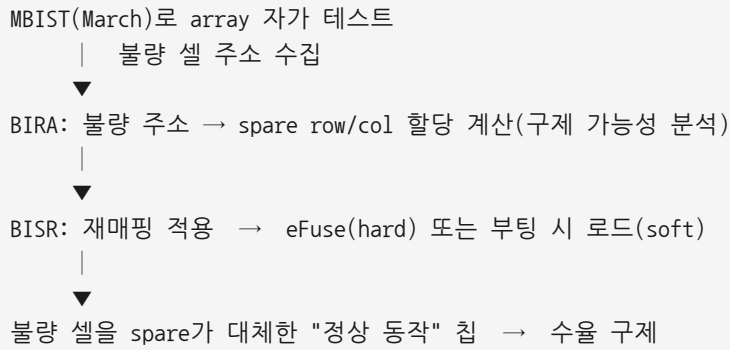
여기가 메모리 양산의 *경제학*이다. 수억 셀 중 몇 개가 불량이라고 칩 전체를 버리면 수율이 처참하다. 그래서 메모리는 처음부터 **여분(spare)** 을 깔아둔다 — **spare row / spare column**. MBIST가 불량 셀의 주소를 찾으면, 그 불량 행/열을 여분 행/열로 **재매핑(remap)** 해 **불량을 우회**한다. 일부 셀이 불량이어도 칩을 살린다. "**메모리는 repair로 yield를 산다**" — 이 한 문장이 메모리 검증의 정수다. 로직엔 없는, 메모리만의 개념이다.

8.4 BIRA / BISR — 칩이 스스로 진단하고 스스로 고친다

repair를 *자동화*한 흐름: - **BIRA(Built-In Redundancy Analysis)**: MBIST가 찾은 불량 주소들을 모아, "이 불량들을 어떤 spare row/column 조합으로 덮을 수 있나"를 *계산*하는 엔진. 불량이 흩어져 있으면 spare를 어떻게 배분해야 최대한 구제되는지가 *조합 최적화* 문제라, 전용 알고리즘이 필요하다. - **BISR(Built-In Self-Repair)**: BIRA의 해를 받아 *실제로* 재매핑을 적용. 그 매핑 정보를 어딘가 영구 저장해야 한다.

저장 방식: - **Hard repair**: 제조 테스트 시점에 불량 주소를 **eFuse**(전기적으로 끊어 영구 기록하는 퓨즈)에 구워 영구 재매핑. 칩 수명 내내 유효. - **Soft repair**: 부팅(power-on)마다 비휘발 저장에서 매핑을 불러와 재구성. 유연하지만 매번 적용.

전체 흐름을 한 그림으로:



8.5 On-die ECC — 미세화가 부른 추가 방어선

미세화로 셀이 작아지면 single-bit error 발생률이 올라간다(retention 악화, 입자성 오류 등). 이걸 array 테스트·repair로 다 잡기엔 한계라, **칩 내부에 ECC(Error Correcting Code)**를 내장해(DDR5 on-die ECC) 동작 중 발생하는 single-bit error를 **실시간으로** 정정한다. SECDED(Single Error Correct, Double Error Detect)가 기본 원리다. HBM은 link/내부 ECC를 추가로 둔다. ECC 인코더/디코더는 **Peri의 디지털 로직** — 즉 설계·검증 대상이다.

🧠 **메모리 브릿지 (이 장 전체)**: MBIST 컨트롤러·BIRA·BISR·ECC 코덱은 전부 **Peri 디지털 RTL**이다. 즉 *지원자의 RTL 검증·DFT·자동화가 직접 닿는 영역*이다. "셀(소자)은 입사 후 채우되, 그 셀을 테스트·repair·정정하는 **디지털 로직**의 검증은 내가 이미 하던 일"이라는 브릿지가 여기서 완성된다. HBM처럼 적층 후 내부 die 접근이 어려운 구조에서는 MBIST·BISR의 가치가 더 커진다 — 외부 테스터가 못 닿는 곳을 칩이 스스로 테스트·repair하기 때문이다.

9. 검증 자동화와 AI — 지원자의 차별점, 그리고 그 철학

검증은 한 번 돌리고 끝이 아니다. 설계가 바뀔 때마다 **전체**를 다시 돌려 회귀(regression)를 막아야 하고, 수천 개 테스트의 결과를 **분류**해야 하며, coverage hole을 추적해야 한다. 이 반복·대량·판단 작업이 검증 비용의 큰 부분이고, 자동화·AI가 들어갈 자리다. 지원자의 핵심 차별점이 정확히 여기다.

Regression: 모든 테스트(directed + random seed들)를 야간·주기적으로 자동 실행해, 어제 고친 게 오늘 안 깨졌는지를 지속 확인한다. 수천 개 잡(job)의 스케줄링·결과 수집·실패 triage가 거대한 자동화 문제다. 지원자의 **Altair FlowTracer로 PI/PD TAT 20%↓**(의존성 그래프·병렬화·자동 trigger) 경험은 이 대량 잡 오케스트레이션 감각과 동형이다.

Triage: regression이 수백 개 fail을 뱉으면, 그게 **진짜 버그 N개가 일으킨 것인지**(같은 원인의 중복)를 분류해야 한다. 지원자의 **CDC report를 register 단위로 클러스터링**(04 문서 2번), **RTL Sign-off Agent의 근거 출처별 신뢰도 스코어링**(04 문서 3번)이 바로 "대량 리포트를 의미 단위로 묶고 우선순위를 매기는" triage 자동화다.

AI 보조 검증: 여기가 지원자만의 영역이다 — **RAG/LLM 기반 RTL Sign-off Agent(false 20%↓, 사업부 1호)**, **CDC 분류 LLM PoC(95% 자동분류)**, **AI Skill 공유 플랫폼(500명+, 조직혁신상)**. 핵심은 **방법**이 아니라 **철학**이다:

"AI는 sign-off를 대체하지 않는다. 사람 판단 *아래*의 보조도구이고, 반드시 신뢰도/근거를 남겨 사람이 검증 가능해야 한다."

이 톤을 면접 내내 일관되게 유지해야 한다. CDC/검증은 *static하게 100% 정확해야* sign-off로 넘어가는 영역이라, AI의 잔여 오류를 신뢰할 근거가 없으면 못 쓴다(지원자가 2년 전 ML 단독 분류가 왜 실패했는지 이미 안다). 그래서 지원자의 설계는 항상 (1) **확실한 건 deterministic 로직으로, 모호한 건 RAG/LLM으로 분기**, (2) **근거 출처별 신뢰도 스코어로 "왜 그렇게 판단했나"를 사람이 검증 가능하게 남긴다**. "AI가 틀리면?"이라는 질문에 흔들리지 않는 일관된 답을 가진 사람이라는 게 강점이다.

 **메모리 브릿지:** 메모리 R&D도 동일한 검증 자동화 수요가 있다 — 방대한 IP, 긴 runtime, 대량 sign-off 리포트. SK하이닉스의 AI Transformation·CAD 확산에 지원자의 "검증 자동화 + 안전한 AI 보조" 패턴이 그대로 기여한다. PIM(GDDR6-AiM)처럼 *메모리 안에 디지털 연산 로직이 들어가는* 제품은 검증 수요가 더 크고, 지원자의 디지털 설계·검증·AI 교집합이 정확히 맞는다.

10. 면접 연결 — 30초 / 1분 골격과 브릿지

각 항목은 *외우는 스크립트가 아니라* 앞 장의 이해를 30초/1분으로 압축하는 골격이다.

10.1 "UVM을 왜 쓰나"

- **30초:** "검증 환경을 *재사용*하기 위해서입니다. agent/driver/monitor/sequencer/scoreboard로 책임을 계층화하고, factory·config_db로 부품을 안 고치고도 행동을 바꿔, 한 번 만든 검증 자산(VIP)을 다른 프로젝트·팀이 그대로 씁니다. constrained-random과 functional coverage를 체계적으로 돌리는 표준 골격입니다."
- **1분 확장:** directed test의 "본 것만 안다" 한계 → CRV로 corner 자동 도달 → 정답을 모르니 scoreboard+SVA가 판정 → coverage-driven으로 "충분히 봤나"를 측정. 이 흐름을 재사용 가능하게 표준화한 게 UVM. **브릿지:** 메모리 컨트롤러는 host active agent + DRAM passive agent, JEDEC 프로토콜을 monitor의 SVA로 감시.

10.2 "Coverage closure란"

- **30초:** "테스트가 통과한 게 아니라, code+functional coverage가 목표치에 도달하고, assertion이 다 pass하고, regression이 안정적으로 green이며, 남은 hole이 directed로 채워지거나 도달 불가로 입증돼 waiver된 상태입니다. '안 본 곳이 없고, 본 곳은 다 맞다'가 성립할 때 sign-off합니다."
- **꼬리 방어:** "coverage 100%면 버그 없음?" → 아니다. coverage는 완전성, 정확성은 checker가 보장. **브릿지:** 메모리 프로토콜의 명령·timing corner cross coverage hole 추적이 검증 자동화의 핵심.

10.3 "LEC vs simulation"

- **30초:** "시뮬레이션은 입력 벡터의 일부만 봅니다. LEC는 두 설계를 compare point로 쪼개 각 logic cone이 *모든 입력 조합에서* 같음을 형식적으로 증명합니다. 등가성 보장엔 LEC가 정답입니다. 합성·DFT 삽입·ECO·restructuring 후 기능 보존 확인에 씁니다."

- **1분 + 경험:** "ARM 코어를 SoC compiler로 재구조화하면 hierarchy가 변형돼 parameter 오류·\ define 소실 위험이 있어 Formality로 등가성을 sign-off했습니다. mapping 파일을 파싱해 compare rule을 자동생성하고, Full EQ(정밀 sign-off)와 Quick EQ(하위 black-box·인터페이스 1차 확인)를 이원화해 1,760만 line을 입력 5→2 상황에서도 검증했습니다. 도구 버그(parameter·\ define)도 재현해 벤더 fix를 유도했습니다." **브릿지:** 메모리 Peri 합성·ECO 정합성에 직접 이식.

10.4 "Scan / ATPG"

- **30초:** "테스트 용이성을 설계 때 넣는 겁니다. FF를 scan FF로 바꿔 shift register로 묶으면, shift로 내부를 임의 값으로 세팅(controllability)하고 capture로 응답을 받아 관찰(observability)합니다. ATPG가 stuck-at/transition fault를 잡는 패턴을 자동 생성하고, compression으로 테스트 시간을 줄입니다."
- **꼬리 방어:** "stuck-at vs transition?" → stuck-at은 고착, 느린 클럭으로도 잡힘. transition은 천이 지연, at-speed 두 클럭 필요. "scan과 CDC 충돌?" → shift 시 도메인이 한 체인에 직렬로 묶여 기능 모드 동기화가 무력화 → 별도 scan 클럭 제어·capture 경로 처리 필요(제 CDC 관점의 접점). **브릿지:** 메모리는 array 결함률↑로 DFT 비중이 큼.

10.5 "MBIST / repair가 왜 메모리에 중요한가"

- **30초:** "수억 셀을 핀에서 일일이 못 보고, 적층 die는 외부 접근도 어렵습니다. 그래서 칩 내부 MBIST 엔진이 March 패턴으로 self-test해 SAF/TF/coupling/decoder fault를 잡습니다. 불량 셀은 spare row/column으로 재매핑(BIRA→BISR, eFuse)해 수율을 구제합니다. 메모리는 repair로 yield를 잡니다 — 로직엔 없는 개념입니다. DDR5는 on-die ECC까지 더합니다."
- **1분 + 브릿지:** MBIST·BIRA·BISR·ECC는 전부 Peri 디지털 로직 → 제 RTL 검증·DFT·자동화가 직접 닿는다. HBM 적층처럼 내부 die 접근이 어려운 구조에서 self-test·self-repair의 가치가 더 크다.

10.6 종합 브릿지 — "삼성 sign-off/검증 자동화 → 메모리 Peri/HBM Base Die 검증"

지원자의 최강 한 문장: "제가 삼성에서 한 일은 RTL이 silicon이 되기까지의 sign-off 관문 — Lint/CDC/RDC, Formality EQ, 검증 자동화 — 을 설계자가 신뢰하고 통과하게 만드는 것이었습니다. 메모리 Peri·HBM Base Die도 정확히 같은 디지털 RTL이고, 같은 관문을 통과합니다. 다른 건 셀(소자) 디테일뿐이고, 그건 입사 후 빠르게 채우겠습니다. 검증의 문제는 같습니다." 이 명제를 모든 검증 답변의 착지점으로 쓴다.

11. 스스로 점검 (정독 후 막힘없이 답하면 통과)

1. 검증이 설계비용의 절반 이상인 이유를 상태공간 폭발 + respin 비대칭 비용 두 축으로 30초에 설명할 수 있는가? 그리고 그것을 내 RDC 87.7%↓ / CDC 95% 분류 숫자로 "shift-left"라 부를 수 있는가?
2. 시뮬레이션 / STA / form(LEC·property) 세 검증 축의 역할 분담을 한 그림으로 그릴 수 있는가? "전수를 보는 건 누구인가?"
3. directed → CRV → coverage-driven의 세 패러다임 전환을 "왜 다음 단계가 필요했나"의 인과로 이어 말할 수 있는가?
4. code coverage 100% ≠ 검증 완료인 이유를, cross coverage 예시(동시 발생 조합)로 설명할 수 있는가? coverage(완전성)와 checker(정확성)가 다른 축임을 분리해 말할 수 있는가?
5. UVM 부품 6개(sequence/sequencer/driver/monitor/scoreboard/agent)의 역할과, factory·config_db가 왜 "재사용"의 열쇠인지 설명할 수 있는가? "Spec 기반 Custom UVM Agent를 만든다"가 무슨 작업인지, 내

IP-XACT 자동생성 경험과 연결되는가?

6. SVA의 immediate vs concurrent, $| \rightarrow$ vs $| \Rightarrow$ 를 구분하고, **formal**이 잘 맞는 문제 유형(작은 제어·프로토콜, unbounded) 과 안 맞는 유형(넓은 데이터패스)을 그 이유와 함께 말할 수 있는가?
7. LEC가 **compare point + logic cone**으로 순차 문제를 조합 문제로 *분해*하는 원리를 설명할 수 있는가? Quick EQ에서 black-box하면 그 안이 검증 안 된다는 지적에, **속도-정확도 의도적 분리**로 방어할 수 있는가?
8. scan의 **shift / capture** 동작을 controllability/observability로 설명하고, **stuck-at**(느린 클럭) vs **transition(at-speed)**의 차이와 scan↔CDC 충돌(shift 시 도메인 직렬화)을 말할 수 있는가?
9. **March 알고리즘이 잡는 결함(SAF/TF/coupling/decoder)** 과, 양방향 훅기로 *coupling*을 잡는 이유를 설명할 수 있는가? BIRA→BISR→eFuse repair 흐름과 "메모리는 repair로 yield를 산다"의 의미를 말할 수 있는가?
10. AI 보조 검증을 말할 때 "**sign-off 대체 아님, 사람 판단 아래 보조 + 신뢰도/근거**" 철학을 일관되게 유지하고, 2년 전 ML 단독 분류가 *왜* 실패했는지(static 100% 정확성 요구 vs 신뢰 근거 부재)를 설명할 수 있는가?
11. 모든 검증 답변을 "**검증의 문제는 같다 — 삼성 sign-off → 메모리 Peri/HBM**" 한 문장으로 착지시킬 수 있는가?

연계: 기능검증 기초 `daily_study/H_verif.md` · 디지털/합성 `daily_study/G_digital.md` · CDC/RDC/STA `daily_study/D_cdc · E_sta` · 경력 방어 `04_resume_selfmastery.md` (4번 EQ·2번 CDC·3번 Agent·11번 SFR) · 질문 은행 `03_interview_job_qbank.md` C4~C8.

concepts/07_검증_DFT_MBIST_완전이해.md · SK하이닉스 설계 전형 준비 자료